Configuring Kubernetes Horizontal Pod Autoscaler (HPA) for Custom Metrics

Document ID: DOC-CLD-510 | Category: Cloud Computing & DevOps | Version: 3.1

1. Beyond CPU and Memory Autoscaling

By default, the Kubernetes Horizontal Pod Autoscaler (HPA) scales replica counts based on observed CPU and Memory utilization provided by the Metrics Server. While sufficient for compute-bound applications, relying solely on hardware metrics is ineffective for event-driven architectures. For example, a background worker pulling from a message queue might use negligible CPU, yet a massive backlog of messages necessitates scaling up. This is achieved by configuring the HPA to respond to **custom metrics** (e.g., HTTP requests per second, queue depth, or active database connections).

2. The Custom Metrics Pipeline

Kubernetes cannot natively query external monitoring systems like Prometheus or Datadog. To use custom metrics, cluster administrators must deploy an adapter—such as the `prometheus-adapter`—which acts as a bridge. The adapter continually scrapes metrics from the monitoring system and exposes them to the Kubernetes API server via the `custom.metrics.k8s.io` or `external.metrics.k8s.io` APIs. Once registered, the HPA controller can query these metrics natively.

3. HPA API Configuration Structure (v2)

When defining an HPA object using the `autoscaling/v2` API, developers define the scaling criteria in the `metrics` array. The configuration supports three primary metric types: `Resource` (native CPU/Memory), `Pods` (metrics describing each pod in the target, which are averaged), and `Object` / `External` (metrics describing a single external entity, like an SQS queue).

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-worker-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-worker
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: External
    external:
      metric:
        name: rabbitmq_queue_messages
        selector:
          matchLabels:
            queue_name: "order_processing"
      target:
        type: AverageValue
        averageValue: 50
```

In this specification, the HPA will monitor the `rabbitmq_queue_messages` metric and scale the `order-worker` pods up or down to maintain an average of 50 queued messages per pod.

4. Managing Scale-Down Stabilization

A common pitfall with custom metrics—particularly queue lengths or bursty HTTP traffic—is "thrashing," where the HPA scales replicas up and down too rapidly. This destabilizes the application and disrupts network routing. To mitigate this, developers should configure the `behavior` block to define stabilization windows. Setting a `scaleDown.stabilizationWindowSeconds` of 300 instructs the HPA controller to wait 5 minutes before removing replicas after the metric drops, ensuring the traffic spike has genuinely subsided rather than momentarily dipped.